

Design Patterns

Proven Solutions to Recurring Problems

Design patterns are named solutions to common problems in software design. They're not code you copy-paste—they're templates for solving problems that you adapt to your specific situation.

The concept was popularized by the “Gang of Four” (GoF)—Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides—in their 1994 book *Design Patterns: Elements of Reusable Object-Oriented Software*. They catalogued 23 patterns drawn from experience, giving developers a shared vocabulary for discussing design.

Patterns aren't invented; they're discovered. They emerge from observing what works across many codebases. Knowing them helps you recognize opportunities and communicate solutions.

Creational Patterns

These patterns deal with object creation.

Pattern	Description
Abstract Factory	Create families of related objects without specifying concrete classes.
Builder	Construct complex objects step by step, separating construction from representation.
Factory Method	Let subclasses decide which class to instantiate.
Prototype	Create new objects by copying an existing object.
Singleton	Ensure a class has only one instance and provide global access to it.

Structural Patterns

These patterns deal with composing classes and objects.

Pattern	Description
Adapter	Convert one interface into another that clients expect.
Bridge	Separate an abstraction from its implementation so both can vary independently.
Composite	Treat individual objects and compositions uniformly as a tree structure.
Decorator	Add responsibilities to objects dynamically without subclassing.
Facade	Provide a simplified interface to a complex subsystem.
Flyweight	Share common state among many objects to reduce memory usage.
Proxy	Provide a placeholder that controls access to another object.

Behavioral Patterns

These patterns deal with communication between objects.

Pattern	Description
Chain of Responsibility	Pass requests along a chain of handlers until one handles it.
Command	Encapsulate a request as an object, enabling undo, queuing, and logging.
Interpreter	Define a grammar and an interpreter for a language.
Iterator	Access elements of a collection sequentially without exposing its structure.
Mediator	Reduce chaotic dependencies by having objects communicate through a central mediator.
Memento	Capture and restore an object's internal state without violating encapsulation.
Observer	Notify multiple objects when one object's state changes.
State	Let an object alter its behavior when its internal state changes.
Strategy	Define a family of algorithms, encapsulate each, and make them interchangeable.

Pattern	Description
Template Method	Define the skeleton of an algorithm, letting subclasses fill in specific steps.
Visitor	Add new operations to objects without modifying their classes.

A Word of Caution

Patterns are tools, not goals. Don't force a pattern where it doesn't fit. Over-applying patterns creates unnecessary complexity—sometimes called “pattern fever.” The best code often uses no recognizable patterns at all; it's just simple and clear.

Learn patterns so you recognize when they genuinely help, not so you can show off.

References

- Erich Gamma et al., *Design Patterns: Elements of Reusable Object-Oriented Software* (1994) — The original “Gang of Four” book.
- Refactoring.Guru - Design Patterns — Excellent visual explanations of all patterns with code examples.
- Joshua Kerievsky, *Refactoring to Patterns* (2004) — How to evolve code toward patterns through refactoring.